Version Control Activity

Code Escape: Collaborative Development Workflow

Capstone Project (Unity, Git and GitHub)

Group Number	IT22
Group Members	Jean Joseph C. Mallari Mark Daniel M. Haguisan Louis Angelo D. Junio Alynus Job C. Tombado
Version-Control Platform	Git + GitHub (GitHub Desktop as the client)
Repository	https://github.com/jjcmall/Code-Escape-IT22
Course / Section	*[Course and section]*
Instructor	*[Instructor]*
Date prepared	8 November 2026

Status: Parts I, II, III, VII and VIII are filled from the actual repository. Mark Daniel M. Haguisan's branch and commits are recorded in Parts IV, VI and IX. The other members' work, the pull requests and the reviews are still to be done and are marked in italics.

Part I. Version-Control Readiness Check

Requirement	Status	Evidence / Action Taken
Repository exists	Complete	GitHub repository `jjcmall/Code-Escape-IT22`; the local copy has `origin` pointing to it.
All active members have appropriate access	Incomplete	*To confirm: in GitHub, Settings > Collaborators, all four members are listed with Write access. Today every one of the 33 commits comes from one account (`jjcmall`), so the other members have not yet committed.*
Main / accepted development branch exists	Complete	`main` (the remote default, `origin/HEAD` points to `origin/main`).
Project can be retrieved by another member	Incomplete	*To confirm: a second member clones the repository with GitHub Desktop (File > Clone repository) and shows it in the evidence.*
Project opens correctly after retrieval	Incomplete	*To confirm: the clone is added in Unity Hub with Editor 6000.5.10f1 (the version in `ProjectSettings/ProjectVersion.txt`) and opens without errors.*
Generated / unnecessary files are handled appropriately	Complete	`.gitignore` (Unity template) excludes `Library/`, `Temp/`, `Logs/`, `UserSettings/`, `obj/`, `Build/`, `*.csproj` and `*.sln`. Checked with `git check-ignore`: `Library`, `Temp`, `Logs` and `UserSettings` are ignored although they exist on disk.
Important project files are tracked	Complete	1672 tracked files: `Assets/` (1621, including 849 `.meta` files), `ProjectSettings/` (29), `Packages/` (`manifest.json`, `packages-lock.json`), `Documentation/` (13), `README.md`.
Change history is available	Complete	33 commits from 11 October 2026 (`git log`, also GitHub Commits page).

Project Title	Code Escape
Version-Control Platform	Git + GitHub
Repository Name	Code-Escape-IT22
Repository Link	https://github.com/jjcmall/Code-Escape-IT22
Main Branch	`main`
Development Platform / Game Engine	Unity 6000.5.10f1 (C#)
Repository Visibility / Access Setting	*[Public or Private: confirm in GitHub Settings. If private, add the instructor as a collaborator.]*

Honest status: the repository is set up correctly. Before this activity it had one contributor (`jjcmall`) and one working branch. Mark Daniel M. Haguisan now has a task branch with commits; the reviews and merges by different members are still to be done.

Part II. Repository and Project Organization

Which project files are version controlled?

- `Assets/`: scripts, scenes, prefabs, art, audio, fonts, data and every `.meta` file. The `.meta` files hold the asset IDs, so they must be committed or references break for the other members.
- `ProjectSettings/` and `Packages/manifest.json` + `packages-lock.json`: the engine settings and package versions, so every member opens the same project.
- `Documentation/` and `README.md`.
- Unity is set to **Force Text** serialization (`m_SerializationMode: 2`), so scenes and prefabs are text (YAML) and Git can show and merge differences.

Which generated files are excluded?

`Library/` (Unity's import cache), `Temp/`, `Logs/`, `obj/`, `Build/`, `UserSettings/` (per-user editor layout), generated `*.csproj` and `*.sln`, and `*.log`. They are rebuilt on each computer, and committing them would cause constant conflicts.

How are large assets handled?

The largest tracked file is `Assets/Animations/Sam/Sam.fbx` (32.7 MB), then a 17.2 MB and a 9.4 MB texture for the same character. No file is over 50 MB, so plain Git is used. `.gitattributes` only has `* text=auto` (line-ending normalisation). Decision: Git LFS is **not needed now**. If a model, audio or video file approaches 50 MB (GitHub warns at 50 MB and blocks at 100 MB), the group will add it to Git LFS before committing it.

How does another member obtain a usable copy?

1. Install Unity Editor 6000.5.10f1 through Unity Hub.
2. In GitHub Desktop, choose File > Clone repository, pick `Code-Escape-IT22` and clone it (or use Unity Hub: Add > Add project from repository > GitHub).
3. In Unity Hub choose Add > Add project from disk and select the cloned folder. The first open is slow because Unity rebuilds `Library/`.
4. Sign in to Unity with the group's agreed account if cloud features need to be tested (see `Documentation/CloudSave.md`).

[Screenshot: the repository on GitHub (file list with Assets, ProjectSettings, Packages) and the .gitignore file. To be added by the group.]

Part III. Branching Plan

`main` is the only accepted version. Every task below starts from the latest `main` and ends in a reviewed merge. The tasks come from the group's real pending work. The member column is a proposal for the group to confirm.

Branch	Purpose	Assigned Member/s	Expected Output
`main`	Accepted / integrated project version	Group	Stable development baseline; changes arrive only by reviewed pull request
`refactor/save-progress`	Move record building and snapshot restoring out of `CloudProgressService`	Mark Daniel M. Haguisan	`ProgressRecordBuilder.cs`, `ProgressSnapshotApplier.cs`, slimmer `CloudProgressService.cs`; architecture report; tests of Part VII of the Architecture Activity
`feature/cloud-accounts`	Username and password accounts with Unity Authentication and Cloud Save	*[Member, proposed]*	Sign-in gate and account menu in `ProgressionUI.cs`, account methods in `CloudProgressService.cs`, `Documentation/CloudSave.md`
`feature/menu-theme`	One shared look for the menus and screens	*[Member, proposed]*	`MenuTheme.cs`, `MainMenuUI.cs`, related prefabs and sprites
`docs/version-control-activity`	This version-control report	Mark Daniel M. Haguisan	Files in `Documentation/VersionControlActivity/`

Naming. `feature/` for new work, `fix/` for defects, `refactor/` for restructuring, `docs/` for documents, in lower case with hyphens. One branch is one task. Branches are deleted after the merge.

> Existing branches: `main`, plus `split-history`, `1`, `backup/initial-commit` and `backup/main-before-split` left over from the first upload on 11 October 2026. They are not part of the plan; the group should delete the extra ones from GitHub after confirming nothing is needed from them.

Part IV. Individual Development Work and Change History

Branches and commits for Mark Daniel M. Haguisan are real and were made on 8 November 2026. The other three members' rows are still empty and must only be filled with their own commits.

Current history (actual)

Author	Commits	Examples (hash, date, message)
jjcmall	33 on `main`	`69731f9` 24 Oct 2026, Add code-driven devices, achievements, bug traps, alarm escape and Signal to Sam `b51c3cc` 20 Oct 2026, Make the game Python only `e1edf6e` 20 Oct 2026, Add sandboxed Python interpreter for running player code
Mark Daniel M. Haguisan	2 on `refactor/save-progress`; 1 on `docs/version-control-activity`	`3ac1680` 8 Nov 2026, Move record building and snapshot restore out of CloudProgressService `77a743a` 8 Nov 2026, Add software architecture activity report for the save progress refactor Docs branch: Add version control activity report (hash on GitHub after publishing)
Other members	0	*No commits yet.*

Good commit messages in this history describe the change. Messages such as "PACKAGES" and "GITIGNORE AND README" are too short, and the group will avoid them from now on.

Per-member record (to be completed)

Member	Branch	Assigned task	Commit hash	Description	File / component
Jean Joseph C. Mallari	*[branch]*	*[task]*	*[hash]*	*[message]*	*[file]*

Member	Branch	Assigned task	Commit hash	Description	File/component
Mark Daniel M. Haguisan	`refactor/save-progress`	Separate record building and snapshot restoring from `CloudProgressService`	`3ac1680`, `77a743a`	Move record building and snapshot restore out of CloudProgressService; Add software architecture activity report	`Cloud/ProgressRecordBuilder.cs`, `Cloud/Progress`

Member	Branch	Assigned task	Commit hash	Description	File/component
					SnapshotApplier.cs, Cloud/CloudProgressService.

Member	Branch	Assigned task	Commit hash	Description	File/component
					cs
Louis Angelo D. Junio	*[branch]*	*[task]*	*[hash]*	*[message]*	*[file]*
Alynus Job C. Tombado	*[branch]*	*[task]*	*[hash]*	*[message]*	*[file]*

Setting the author. Each member must commit under their own GitHub account. In GitHub Desktop: File > Options > Git, set the name and the email that is added to their GitHub account. Otherwise the history shows the wrong person.

[Screenshot: GitHub commit history of each member's branch, showing author, message and hash.]

Part V. Change Review

Every branch is merged only through a Pull Request that another member reviews. The developer never reviews their own pull request.

Pull Request template (used for every PR)

```
Title: short description of the change (example: Move record building into ProgressRecordBuilder)
What was changed:
Why the change was needed:
Feature / component it supports:
How it was tested (Unity version, steps, result):
Files that should be reviewed first:
```

Pull Request description for `refactor/save-progress` (ready to paste)

```
Title: Move record building and snapshot restore out of CloudProgressService
What was changed: Added ProgressRecordBuilder (game state to progression record and snapshot, score rule,
achievements) and ProgressSnapshotApplier (restore a snapshot into the game). CloudProgressService now calls
them and keeps connection, accounts and save timing. The same files also carry the username and password
account changes made at the same time.
Why: CloudProgressService mixed networking, account actions, save timing, the score rule and achievement
rules, so a rule change meant editing the class that talks to Cloud Save.
Feature / component: Save Player Progress (Assets/Scripts/Cloud).
How it was tested: Unity 6000.5.10f1 compiled with no errors. In Play Mode, 8 of 8 save tests passed (solve a
challenge, real save to Cloud Save and reload, nothing-changed, failed save and retry, level clearing, score
rule, load, old record loads). See Documentation/ArchitectureActivity, Part VII.
Review first: Cloud/ProgressRecordBuilder.cs, Cloud/ProgressSnapshotApplier.cs,
Cloud/CloudProgressService.cs.
```

Reviewer checklist

Check	Result (Yes / No / Comment)
The change belongs to the assigned task	
No unrelated files were added (for example scene changes made by accident, `Library/`, `UserSettings/`)	
The code is understandable	
It follows the agreed architecture (view, controller, rules, data service)	
It does not duplicate logic that already exists	
It does not break another component	
It was tested and the Unity Console has no errors	
It is ready to merge	

Review log (to be completed)

Pull Request	Developer	Reviewer	Feedback	Outcome
[PR link]				*Approved / Changes requested*
[PR link]				
[PR link]				
[PR link]				

If changes are requested: the reviewer comments on the line, the developer fixes it with a new commit on the same branch, the pull request updates itself, and the reviewer checks again before approving.

[Screenshot: a Pull Request with the description, the reviewer's comment and the Approved status.]

Part VI. Merging and Team Synchronization

Merge record (to be completed)

Change	Developer	Reviewer	Review Status	Merge Status	Verification
Move record building and snapshot restore out of CloudProgressService (`refactor/save-progress`)	Mark Daniel M. Haguisan	*[another member]*	Not yet reviewed	Not merged	Compiles in Unity; 8 of 8 save tests passed. After the merge: pull `main` and test again.
[PR title]					
[PR title]					
[PR title]					

After each merge

- The merged feature is visible on `main` on GitHub (the merged pull request and the commits).
- One member pulls `main` (GitHub Desktop: Fetch origin, then Pull origin), opens the project in Unity 6000.5.10f1 and checks that the Console has no errors.
- That member plays the main flow (menu, sign in, a challenge, level map) to check that existing features still work, and notes the result in the pull request.
- Every other member pulls the new `main` and merges or rebases it into their branch before continuing (in GitHub Desktop: Branch > Update from main).

[Screenshot: the merged pull request, and `main` commit history after the merge.]

Part VII. Merge Conflict and Game-Asset Collaboration

Example from our own project: two members change `Assets/Scripts/Cloud/CloudProgressService.cs` in different branches. One adds the username and password sign-in (`feature/cloud-accounts`); the other moves the record-building code into `ProgressRecordBuilder` (`refactor/save-progress`). Both edit the same file, and the second one to merge sees a conflict.

1. What could cause the conflict? Two branches changed the same lines (or neighbouring lines) of the same script, or both changed the same scene or prefab, for example `00_MainMenu.unity`. It also happens when a member keeps working from an old `main` for days.

2. How does Git and GitHub identify or prevent it? Git cannot merge overlapping changes by itself. GitHub shows "This branch has conflicts that must be resolved" on the pull request, and GitHub Desktop lists the conflicted files and marks them with `<<<<<<<`, `=======`, `>>>>>>>`. We reduce the risk by keeping branches small and short, updating from `main` before starting and before opening a pull request, and telling the group before editing a scene or prefab (only one member edits a given scene at a time).

3. How should the team resolve it?

- Talk first: both versions may be valid work, so the two developers decide together.
- Update the branch from `main`. In the conflicted script, keep both changes and remove the conflict markers. For a scene or prefab, keep one version and redo the other member's edit in the Unity editor, because merged YAML can be invalid.
- Commit the merge, push, and ask the reviewer to check the resolved file.

4. What should be tested after resolution? Unity opens the project and the Console has no compile errors; the project plays from the main menu; both features still work (sign in and save, and the refactored save path); and the pull request diff contains only the intended files.

Game assets. Scenes and prefabs are saved as text, which helps, but they still conflict easily. Rules: one member owns a scene or prefab at a time; models, textures and audio are binary, so they cannot be merged and the later version simply replaces the earlier one. For those files, tell the group before replacing one, and use Git LFS if a file grows toward 50 MB. Git does not have file locking by default, so the agreement between members acts as the lock.

Part VIII. Team Collaborative Workflow

Before starting development

```
Fetch origin and pull the latest main
-> confirm the assigned task with the group
-> create a branch from main (Branch > New branch, name feature/... or fix/...)
```

During development

```
Make a small change
-> test in Unity (Console has no errors, the feature works)
-> commit with a meaningful message (what and why), under your own account
-> push the branch to GitHub often (Publish branch, then Push origin)
-> update from main when others have merged
```

Before integration

```
Finish the task and test it
-> open a Pull Request with the template (what, why, component, how tested)
-> a different member reviews with the checklist
-> developer fixes issues with new commits if changes are requested
```

After approval

```
Merge the Pull Request on GitHub (a merge commit, so each member's commits stay visible)
-> delete the branch
-> everyone fetches and pulls the new main
-> one member tests the integrated project in Unity
-> continue with the next task
```

Team rules

- No direct commits to `main`. The group turns on branch protection in GitHub (Settings > Branches > Add rule for `main`: require a pull request and at least 1 approval).
- One branch per task, merged within a few days.
- Do not commit `Library/`, `Temp/`, `Logs/` or `UserSettings/` (already ignored). Do not commit passwords, keys or tokens.
- Commit messages say what changed. Avoid "update", "final", "done" or "test".
- Announce in the group chat before editing a scene, a prefab or a large asset.

Relevant Version-Control Evidence

No.	Evidence	Status
1	GitHub repository page with the file list and the repository link	*To be added*
2	Settings > Collaborators showing all four members (and the instructor if private)	*To be added*
3	`.gitignore` on GitHub, and the clone on a second computer	*To be added*
4	Branch list on GitHub (`main` and the task branches)	*To be added*
5	Each member's commit (hash, author, message, files)	*To be added*
6	Pull requests with the description, reviewer comments and approval	*To be added*
7	Merged pull requests and `main` history after the merge	*To be added*
8	Branch protection rule for `main`	*To be added*

[Screenshot area]

Part IX. Individual Contribution Record

Each student submits their own PDF with their own record filled in with real work. The filename is Lastname_GroupNumber_VersionControl.pdf (for example Mallari_Group22_VersionControl.pdf). Keep only your own record, or fill all four if the group shares one file and each student then submits the file with their own part completed.

Member 1	
Student Name	Jean Joseph C. Mallari
Group Number	IT22
Version-Control Platform Used	Git + GitHub
Username / Account	*[GitHub username]*
Branch Name	*[branch]*
Assigned Task	*[task]*
Commit Hash	*[hash]*
Relevant File or Component	*[file]*
Pull Request Reference	*[PR link]*
Reviewer	*[name]*
Final Status	*Merged / Requires Revision / In Progress*
What I learned (about 100 words)	*[answer]*

Member 2	
Student Name	Mark Daniel M. Haguisan
Group Number	IT22
Version-Control Platform Used	Git + GitHub
Username / Account	*[GitHub username]. Commit identity: Mark Daniel M. Haguisan, c23-1182-636@uphsl.edu.ph*
Branch Name	`refactor/save-progress` (also `docs/version-control-activity`)
Assigned Task	Refactor the save progress path: separate record building and snapshot restoring from `CloudProgressService`, and document it
Commit Hash	`3ac1680` (refactor), `77a743a` (architecture report)
Relevant File or Component	`Assets/Scripts/Cloud/ProgressRecordBuilder.cs`, `ProgressSnapshotApplier.cs`, `CloudProgressService.cs`
Pull Request Reference	*[PR link, after the branch is published and the PR is opened]*
Reviewer	*[another member]*
Final Status	In Progress (committed; publishing, review and merge still to be done)

What I learned (about 100 words)	*[Write this in your own words: what you learned about branches, commits and reviews, and how you will use them in the remaining capstone work.]*

Member 3	
Student Name	Louis Angelo D. Junio
Group Number	IT22
Version-Control Platform Used	Git + GitHub
Username / Account	*[GitHub username]*
Branch Name	*[branch]*
Assigned Task	*[task]*
Commit Hash	*[hash]*
Relevant File or Component	*[file]*
Pull Request Reference	*[PR link]*
Reviewer	*[name]*
Final Status	*Merged / Requires Revision / In Progress*
What I learned (about 100 words)	*[answer]*

Member 4	
Student Name	Alynus Job C. Tombado
Group Number	IT22
Version-Control Platform Used	Git + GitHub
Username / Account	*[GitHub username]*
Branch Name	*[branch]*
Assigned Task	*[task]*
Commit Hash	*[hash]*
Relevant File or Component	*[file]*
Pull Request Reference	*[PR link]*
Reviewer	*[name]*
Final Status	*Merged / Requires Revision / In Progress*
What I learned (about 100 words)	*[answer]*